

Lekce 17: Geometrie I - struktura geometrií

Python pro GIS - Čtení a zápis objektů Geometry

Vojtěch Barták, FŽP ČZU Praha

2025-12-10

Table of contents

1	Úvod	3
1.1	Data	3
2	Struktura geometrických objektů	3
2.1	Bodové geometrie	4
2.1.1	Struktura jednoduché bodové geometrie	4
2.1.2	Různé způsoby přístupu k souřadnicím	5
2.1.3	Vlastnosti objektu Point	5
2.1.4	Multipart bodové geometrie	6
2.1.5	Zjištění typu geometrie	7
2.2	Liniové geometrie	7
2.2.1	Struktura liniové geometrie	7
2.2.2	Čtení lomových bodů linie	8
2.2.3	Zjištění počtu lomových bodů	8
2.2.4	Extrakce prvního a posledního bodu	9
2.3	Polygonové geometrie	9
2.3.1	Struktura polygonové geometrie	10
2.3.2	Čtení vrcholů polygonu	10
2.3.3	Multipart polygony	11
2.3.4	Centroid a extent	12
2.4	Úlohy	12
3	Tvorba nových geometrií	13
3.1	Tvorba bodové geometrie	13
3.1.1	Základní způsob	13
3.1.2	Vložení bodu do feature class	13
3.1.3	Kratší způsob pomocí SHAPE@XY	14

3.1.4	Import bodů z CSV souboru	14
3.2	Tvorba liniové geometrie	15
3.2.1	Jednoduchá linie	15
3.2.2	Vložení linie do feature class	16
3.2.3	Multipart linie	17
3.3	Tvorba polygonové geometrie	17
3.3.1	Jednoduchý polygon	17
3.3.2	Vložení polygonu do feature class	18
3.3.3	Polygon s dírou (vnitřním ringem)	19
3.3.4	Multipart polygon	20
3.4	Úlohy	21
4	Tahák - časté vzory použití	21
4.1	Čtení geometrií	21
4.1.1	Výpis všech souřadnic bodové vrstvy	21
4.1.2	Počítání lomových bodů linie	21
4.1.3	Extrakce hraničních bodů polygonu	22
4.2	Vytváření geometrií	22
4.2.1	Vytvoření bodu pomocí SHAPE@XY	22
4.2.2	Vytvoření bodu pomocí Point objektu	22
4.2.3	Vytvoření linie ze seznamu souřadnic	22
4.2.4	Vytvoření polygonu ze seznamu souřadnic	23
4.3	Konverze mezi geometriemi	23
4.3.1	Převod centroidů polygonů na body	23
4.3.2	Převod linií na počáteční a koncové body	23
4.4	Import dat	24
4.4.1	Import bodů z CSV	24
4.4.2	Import linií z textového souboru	24
4.5	Generování geometrií	25
4.5.1	Vytvoření pravidelné mřížky bodů	25
4.5.2	Vytvoření pravidelného mnohoúhelníku	25
4.5.3	Vytvoření mřížky polygonů	26
4.6	Zpracování chyb	27
4.6.1	Kontrola typu geometrie	27
4.6.2	Ošetření neplatných geometrií	27
4.6.3	Kontrola uzavření polygonu	27
4.7	Práce se souřadnicovými systémy	28
4.7.1	Specifikace SR při vytváření feature class	28
4.7.2	Transformace souřadnic	28

1 Úvod

V předchozích lekcích jsme pracovali s atributovými daty v tabulkách. Nyní se zaměříme na **geometrickou složku** geoprvků - na objekty reprezentující prostorové tvary jako body, linie a polygony.

Po absolvování lekce budete umět:

- Porozumět hierarchické struktuře geometrických objektů v **arcpy**
- Číst souřadnice bodů, lomových bodů linií a vrcholů polygonů
- Pracovat s jednoduchými i vícedílnými (multipart) geometriemi
- Vytvářet nové geometrické objekty programově
- Vkládat vytvořené geometrie do feature class pomocí kurzorů

i Proč pracovat s geometriemi přímo?

Přímá práce s geometriemi umožňuje:

- Vytvářet nové geoprvky programově (bez ručního digitalizování)
- Analyzovat a upravovat existující geometrie
- Importovat prostorová data z nestandardních formátů
- Provádět vlastní geometrické výpočty a transformace

1.1 Data

V lekci budeme pracovat s následujícími daty:

- Bodová vrstva měst **Sidla_body.shp** z datasetu **Kraje_sidla**
- Liniová vrstva silnic (můžete použít vlastní data nebo stáhnout testovací data)
- Polygonová vrstva krajů **Kraje.shp** z datasetu **Kraje_sidla**

2 Struktura geometrických objektů

Geometrie v **arcpy** mají **hierarchickou strukturu** postavenou na několika základních třídách:

1. **arcpy.Point** - reprezentuje jeden bod (vertex) se souřadnicemi X, Y (a volitelně Z, M)
2. **arcpy.Array** - kolekce bodů nebo kolekcí bodů (posloupnost)
3. **arcpy.Geometry** - základní třída pro všechny geometrie
 - **arcpy.PointGeometry** - bodová geometrie
 - **arcpy.Multipoint** - multipart bodová geometrie

- `arcpy.Polyline` - liniová geometrie
- `arcpy.Polygon` - polygonová geometrie

💡 Analogie se strukturou dat

Můžete si to představit jako:

- `Point` = jednotlivé číslo
- `Array` = seznam čísel
- `Geometry` = komplexní struktura (objekt) obsahující seznamy čísel s dalšími vlastnostmi

2.1 Bodové geometrie

Bodové geometrie (`PointGeometry`) jsou na první pohled nejjednodušší typ geometrie. Každý bodový geoprvek obvykle obsahuje jeden objekt `Point` se souřadnicemi X a Y. Existují však i **multipart bodové geometrie**, které obsahují více oddělených bodů v rámci jednoho záznamu v atributové tabulce.

2.1.1 Struktura jednoduché bodové geometrie

U běžných (single-part) bodových geometrií je struktura přímočará. Když z kurzoru načteme pole `SHAPE@`, získáme objekt `PointGeometry`. Z něj pak můžeme extrahovat samotný bod pomocí vlastnosti `firstPoint`, která vrací objekt `Point` s konkrétními souřadnicemi.

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

with arcpy.da.SearchCursor("Sidla_body.shp", ["SHAPE@", "NAZEV"]) as cursor:
    for row in cursor:
        point_geom = row[0] # Objekt PointGeometry
        point = point_geom.firstPoint # Objekt Point

        # Výpis souřadnic
        print(f"{row[1]}: X={point.X:.2f}, Y={point.Y:.2f}")
```

V této ukázce vidíme základní postup: načteme celou geometrii (`row[0]`), což je objekt `PointGeometry`, a z něj pak získáme konkrétní bod (`firstPoint`). Ten má vlastnosti `X` a `Y` s konkrétními souřadnicemi.

2.1.2 Různé způsoby přístupu k souřadnicím

ArcPy poskytuje několik zkratk pro přístup k souřadnicím bodů, které jsou v mnoha situacích pohodlnější než cesta přes `firstPoint`. Tyto zkratky se používají přímo při specifikaci polí v kurzoru.

```
with arcpy.da.SearchCursor("Sidla_body.shp", ["SHAPE@", "SHAPE@XY", "SHAPE@X", "SHAPE@Y", "N"]) as cursor:
    for row in cursor:
        # Způsob 1: Přes objekt Geometry a Point
        geom = row[0]
        point = geom.firstPoint
        x1, y1 = point.X, point.Y

        # Způsob 2: Přes SHAPE@XY (vrací tuple)
        x2, y2 = row[1]

        # Způsob 3: Přes SHAPE@X a SHAPE@Y
        x3, y3 = row[2], row[3]

        print(f"{row[4]}: ({x1}, {y1})")
```

Tento příklad demonstruje tři ekvivalentní způsoby, jak získat souřadnice téhož bodu. První způsob (`SHAPE@ + firstPoint`) dává nejplnější přístup k objektu `Point`, druhý způsob (`SHAPE@XY`) vrací přímo tuple se souřadnicemi, třetí způsob (`SHAPE@X` a `SHAPE@Y`) vrací jednotlivé souřadnice jako samostatné hodnoty.

💡 Který způsob použít?

- `SHAPE@XY` - nejrychlejší a nejčastější volba, vrací tuple (x, y)
- `SHAPE@X` a `SHAPE@Y` - když potřebujete jen jednu souřadnici nebo když píšete kód, kde je jasné, že pracujete odděleně s X a Y
- `SHAPE@ + firstPoint` - když potřebujete plný přístup k objektu `Point` (např. pro Z či M souřadnice, nebo pro práci s multipart geometriemi)

2.1.3 Vlastnosti objektu Point

Objekt `Point` uchovává nejen povinné souřadnice X a Y, ale může obsahovat i další informace. U 3D dat to je souřadnice Z (nadmořská výška nebo jiná vertikální dimenze), u lineárně referenčních dat může být přítomna hodnota M (měřená vzdálenost podél linie). Pokud tyto dimenze nejsou použity, příslušné vlastnosti mají hodnotu `None`.

```

with arcpy.da.SearchCursor("Sidla_body.shp", ["SHAPE@"]) as cursor:
    row = cursor.next()
    point = row[0].firstPoint

    print(f"X: {point.X}")
    print(f"Y: {point.Y}")
    print(f"Z: {point.Z}") # None pokud není 3D
    print(f"M: {point.M}") # None pokud není M hodnota
    print(f"ID: {point.ID}")

```

V tomto příkladu používáme metodu `cursor.next()` k načtení pouze prvního záznamu, což je užitečné, když chceme prozkoumat strukturu dat bez procházení všech záznamů. Vlastnosti Z a M budou mít hodnotu `None`, pokud data nejsou trojrozměrná nebo neobsahují měřené hodnoty.

2.1.4 Multipart bodové geometrie

Ne všechny bodové geometrie obsahují jen jeden bod. Občas se setkáme s **multipart bodovými geometriemi**, kde jeden záznam v atributové tabulce reprezentuje více oddělených bodů. Typickým příkladem může být měřená lokalita s více měřicími body, které logicky patří k jednomu objektu. Multipart bodová geometrie je reprezentována objektem `Multipoint`.

U multipart geometrií už nestačí `firstPoint` - potřebujeme procházet všechny části geometrie. Objekt `Multipoint` je k tomu účelu uzpůsoben, lze jej totiž procázet v cyklu přes jednotlivé body.

```

with arcpy.da.SearchCursor("Sidla_body.shp", ["SHAPE@", "NAZEV"]) as cursor:
    for row in cursor:
        point_geom = row[0]

        if point_geom.isMultipoint:
            print(f"Počet bodů: {point_geom.pointCount}")

            # Procházení všech bodů multipart prvku
            for point in point_geom:
                print(f" Bod: ({point.X:.2f}, {point.Y:.2f})")

```

Zde vidíme klíčový moment: když procházíme geometrii cyklem (`for point in point_geom`), získáváme objekty `Point`, které obsahují jednotlivé body.

2.1.5 Zjištění typu geometrie

Někdy potřebujeme zjistit, zda je geometrie multipart, aniž bychom procházeli všechny její části. K tomu slouží vlastnost `isMultipart` a vlastnost `partCount`, která říká počet částí geometrie.

```
with arcpy.da.SearchCursor("Sidla_body.shp", ["SHAPE@", "NAZEV"]) as cursor:
    for row in cursor:
        geom = row[0]

        print(f"{row[1]}:")
        print(f"   Je multipart: {geom.isMultipart}")
        print(f"   Počet částí: {geom.partCount}")
        print(f"   Počet bodů celkem: {geom.pointCount}")
        print(f"   Typ: {geom.type}")
```

Vlastnost `type` vrací textový řetězec identifikující typ geometrie - pro bodové geometrie to bude `"point"`, pro multipart body `"multipoint"`. To je užitečné, když pracujeme s různými typy geometrií ve stejném kódu a potřebujeme rozlišit, s čím aktuálně pracujeme.

2.2 Liniové geometrie

Linie (Polyline) jsou složitější než body. Skládají se z **posloupnosti bodů** (lomových bodů), které dohromady tvoří čáru. Každý lomový bod je objekt `Point` a celá posloupnost je uložena v objektu `Array`. Linie tak využívá hierarchii `Point` → `Array` → `Polyline`.

2.2.1 Struktura liniové geometrie

Když z kurzoru načteme liniovou geometrii, získáme objekt `Polyline`, který má řadu užitečných vlastností. Kromě samotné posloupnosti bodů můžeme přímo zjistit délku linie, počet částí (pokud jde o multipart feature) a celkový počet lomových bodů.

```
with arcpy.da.SearchCursor("Silnice.shp", ["SHAPE@"]) as cursor:
    row = next(cursor)
    line = row[0]

    # Základní vlastnosti linie
    print(f"Délka: {line.length:.2f} m")
    print(f"Počet částí: {line.partCount}")
    print(f"Počet bodů: {line.pointCount}")
```

Tyto vlastnosti jsou velmi užitečné pro rychlou analýzu dat bez nutnosti procházet všechny body manuálně. Délka je vypočítána v jednotkách souřadnicového systému dat (typicky metry u projektovaných souřadnicových systémů).

2.2.2 Čtení lomových bodů linie

Pro detailní práci s linií potřebujeme přístup k jednotlivým lomovým bodům. Linií procházíme dvěma vnořenými cykly: vnější cyklus prochází části (parts) linie, vnitřní cyklus prochází body v každé části. U jednoduchých linií (single-part) bude vnější cyklus proveden pouze jednou.

```
with arcpy.da.SearchCursor("Silnice.shp", ["SHAPE@", "NAZEV"]) as cursor:
    for row in cursor:
        line = row[0]
        print(f"\nLinie: {row[1]}")

        # Procházení všech částí linie
        for part in line:

            # Procházení bodů v části
            for point in part:
                print(f"{point.X:.2f}, {point.Y:.2f}")
```

i Co jsou části (parts)?

Jedna feature může obsahovat více oddělených částí. Například dálnice D1 může být rozdělena na několik úseků (parts), které spolu fyzicky nesousedí. Každá část je samostatná linie s vlastní posloupností bodů. V atributové tabulce to zůstává jediným záznamem, ale geometricky jde o více oddělených linií.

2.2.3 Zjištění počtu lomových bodů

Pokud nás zajímá pouze celkový počet lomových bodů bez potřeby je všechny procházet, můžeme použít vlastnost `pointCount`. Někdy ale potřebujeme spočítat body manuálně, například když filtrujeme určité body nebo když pracujeme s multipart features a chceme znát počty bodů v jednotlivých částech.

```
with arcpy.da.SearchCursor("Silnice.shp", ["SHAPE@", "NAZEV"]) as cursor:
    for row in cursor:
        line = row[0]
        total_points = 0
```

```

for part in line:
    part_points = len([p for p in part])
    total_points += part_points

print(f"{row[1]}: {total_points} lomových bodů")

```

List comprehension `[p for p in part]` vytváří seznam všech nenulových bodů v části. Funkce `len()` pak spočítá prvky tohoto seznamu.

2.2.4 Extrakce prvního a posledního bodu

Pro některé analýzy potřebujeme znát pouze koncové body linie - například když testujeme konektivitu sítě nebo když hledáme, kde linie začíná a končí. ArcPy poskytuje vlastnosti `firstPoint` a `lastPoint`, které vrací tyto krajní body přímo, bez nutnosti procházet všechny lomové body.

```

with arcpy.da.SearchCursor("Silnice.shp", ["SHAPE@"]) as cursor:
    for row in cursor:
        line = row[0]

        # První a poslední bod celé linie
        first_point = line.firstPoint
        last_point = line.lastPoint

        print(f"Začátek: ({first_point.X:.2f}, {first_point.Y:.2f})")
        print(f"Konec: ({last_point.X:.2f}, {last_point.Y:.2f})")

```

U multipart linií tyto vlastnosti vrací první bod první části a poslední bod poslední části. Pokud potřebujete koncové body každé části zvlášť, musíte procházet části cyklem a pro každou část extrahovat její první a poslední prvek.

2.3 Polygonové geometrie

Polygony (Polygon) jsou nejsložitější geometrický typ. Podobně jako linie se skládají z posloupnosti bodů, ale tyto body tvoří **uzavřený tvar** - první a poslední bod jsou stejné. Navíc polygony mohou mít **vnitřní hranice** (díry, holes), což vytváří další úroveň složitosti ve struktuře dat.

2.3.1 Struktura polygonové geometrie

Objekt Polygon poskytuje přímý přístup k základním metrickým vlastnostem. Kromě obvodu (který je analogický k délce linie) máme k dispozici i plochu. Stejně jako u linií můžeme zjistit počet částí a celkový počet vrcholů.

```
with arcpy.da.SearchCursor("Kraje.shp", ["SHAPE@", "NAZEV"]) as cursor:
    row = next(cursor)
    polygon = row[0]

    # Základní vlastnosti polygonu
    print(f"Plocha: {polygon.area:.2f} m²")
    print(f"Obvod: {polygon.length:.2f} m")
    print(f"Počet částí: {polygon.partCount}")
    print(f"Počet bodů: {polygon.pointCount}")
```

Plocha je vypočítána v jednotkách odpovídajících kvadrátu jednotky souřadnicového systému (obvykle m²). Vlastnost `length` u polygonů představuje obvod - součet délek všech hran včetně případných vnitřních hranic.

2.3.2 Čtení vrcholů polygonu

Procházení vrcholů polygonu je podobné jako u linií, ale s důležitým rozdílem: posloupnost bodů může obsahovat hodnoty `None`, které signalizují přechod od vnější hranice k vnitřní hranici (díře). První posloupnost bodů vždy definuje vnější hranici polygonu, po `None` následuje posloupnost definující první vnitřní hranici, po další `None` druhou vnitřní hranici, atd.

```
with arcpy.da.SearchCursor("Kraje.shp", ["SHAPE@", "NAZEV"]) as cursor:
    for row in cursor:
        polygon = row[0]
        print(f"\nPolygon: {row[1]}")

        # Procházení všech částí polygonu
        for part_num, part in enumerate(polygon):
            print(f" Část {part_num}:")

            # Sledování, zda jsme ve vnější nebo vnitřní hranici
            is_exterior = True
            for point in part:
                if point:
                    if is_exterior:
```

```

        print(f"    Vnější hranice: ({point.X:.2f}, {point.Y:.2f})")
    else:
        print(f"    Vnitřní hranice: ({point.X:.2f}, {point.Y:.2f})")
    else:
        # None signalizuje přechod na vnitřní ring
        print(f"    --- Vnitřní ring ---")
        is_exterior = False

```

V tomto kódu sledujeme pomocí proměnné `is_exterior`, zda právě procházíme vnější či vnitřní hranici. Potkáme-li hodnotu `None`, přepneme flag na `False` a víme, že následující body patří k díře v polygonu.

i Vnitřní ringy (díry v polygonech)

Polygon může obsahovat díry - například jezero uprostřed kraje nebo park uprostřed zastavěného území. Tyto díry jsou reprezentovány jako **vnitřní ringy**. V posloupnosti bodů je vnitřní ring oddělen hodnotou `None`.

Struktura tedy vypadá takto: - Vnější hranice: posloupnost bodů definující obrys polygonu - `None` - odděluje vnější a vnitřní hranici - Vnitřní hranice: posloupnost bodů definující díru v polygonu - Případně další `None` a další vnitřní hranice...

Vnitřní ringy jsou orientovány opačně než vnější hranice - pokud vnější hranice běží proti směru hodinových ručiček, vnitřní ringy běží po směru hodinových ručiček (nebo naopak).

2.3.3 Multipart polygons

Multipart polygonové geometrie obsahují více oddělených polygonů v rámci jednoho záznamu. To je užitečné například pro reprezentaci ostrůvků patřících k jednomu státu nebo pro parcely, které jsou fyzicky oddělené, ale administrativně tvoří celek.

```

with arcpy.da.SearchCursor("Kraje.shp", ["SHAPE@", "NAZEV"]) as cursor:
    for row in cursor:
        polygon = row[0]

        if polygon.isMultipart:
            print(f"{row[1]} je multipart polygon s {polygon.partCount} částmi")

            for part_num, part in enumerate(polygon):
                print(f"    Část {part_num}")
        else:
            print(f"{row[1]} je jednoduchý polygon")

```

Každá část multipart polygonu může sama obsahovat vnitřní ringy. Struktura tak může být: Část 1 (vnější + díry) → Část 2 (vnější + díry) → ... Všechny tyto části stále patří k jednomu záznamu v atributové tabulce.

2.3.4 Centroid a extent

Pro mnoho analýz nepotřebujeme znát všechny vrcholy polygonu, ale postačí nám některé odvozené vlastnosti. Centroid (těžiště) je bod reprezentující “střed” polygonu, extent (ohraničující obdélník) definuje minimální a maximální souřadnice X a Y polygonu.

```
with arcpy.da.SearchCursor("Kraje.shp", ["SHAPE@", "NAZEV"]) as cursor:
    for row in cursor:
        polygon = row[0]

        # Centroid (těžiště) polygonu
        centroid = polygon.centroid
        print(f"{row[1]} - centroid: ({centroid.X:.2f}, {centroid.Y:.2f})")

        # Ohraničující obdélník (extent)
        extent = polygon.extent
        print(f"    Extent: {extent.XMin:.2f}, {extent.YMin:.2f}, {extent.XMax:.2f}, {extent.YMax:.2f}")
```

Centroid je užitečný například pro umístění popisků na mapě nebo pro výpočet vzdáleností mezi polygony. Extent se často používá pro rychlé prostorové dotazy - dvě feature se nemohou překrývat, pokud se jejich extenty nepřekrývají, což je mnohem rychlejší test než detailní geometrická analýza.

2.4 Úlohy

Úloha 17.1. Napište skript, který projde všechny obce v `Sidla_body.shp` a vypíše ty, které mají souřadnici Y větší než 5900000.

Úloha 17.2. Pro liniovou vrstvu vypočítejte průměrný počet lomových bodů na jednu linii.

Úloha 17.3. Vytvořte skript, který pro každý kraj vypíše počet vrcholů jeho polygonu a identifikuje kraj s nejvíce vrcholy.

Úloha 17.4. Pro polygonovou vrstvu krajů vypočítejte vzdálenost mezi centroidem každého kraje a bodem [0, 0].

Úloha 17.5. Napište funkci, která pro daný polygon určí, zda obsahuje vnitřní ringy (díry).

3 Tvorba nových geometrií

Nyní se naučíme vytvářet geometrické objekty od začátku - programově, bez ručního digitalizování. To je užitečné například při importu dat z nestandardních formátů nebo při generování testovacích dat.

3.1 Tvorba bodové geometrie

Vytvoření bodové geometrie je nejjednodušší - stačí zadat souřadnice X a Y.

3.1.1 Základní způsob

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Vytvoření objektu Point
point = arcpy.Point(500000, 6000000)

# Vytvoření PointGeometry
point_geometry = arcpy.PointGeometry(point)

print(f"Vytvořený bod: ({point_geometry.firstPoint.X}, {point_geometry.firstPoint.Y})")
```

3.1.2 Vložení bodu do feature class

```
# Nejprve musíme vytvořit prázdnou feature class
output_fc = "Nove_body.shp"

# Vytvoření prázdné bodové vrstvy
arcpy.management.CreateFeatureclass(
    arcpy.env.workspace,
    output_fc,
    "POINT",
    spatial_reference=arcpy.SpatialReference(5514) # S-JTSK
)

# Přidání atributového pole
```

```

arcpy.management.AddField(output_fc, "NAZEV", "TEXT", field_length=50)

# Vytvoření a vložení bodů
body = [
    (500000, 6000000, "Bod 1"),
    (510000, 6010000, "Bod 2"),
    (520000, 6020000, "Bod 3")
]

with arcpy.da.InsertCursor(output_fc, ["SHAPE@", "NAZEV"]) as cursor:
    for x, y, nazev in body:
        # Vytvoření Point a PointGeometry objektů
        point = arcpy.Point(x, y)
        point_geom = arcpy.PointGeometry(point)
        cursor.insertRow([point_geom, nazev])

```

3.1.3 Kratší způsob pomocí SHAPE@XY

```

# Stejný výsledek s kratším kódem
with arcpy.da.InsertCursor(output_fc, ["SHAPE@XY", "NAZEV"]) as cursor:
    for x, y, nazev in body:
        cursor.insertRow([(x, y), nazev])

```

SHAPE@XY vs SHAPE@

Pro jednoduché bodové geometrie používejte **SHAPE@XY** - je to rychlejší a jednodušší. **SHAPE@** použijte, když potřebujete pracovat s Z nebo M souřadnicemi.

3.1.4 Import bodů z CSV souboru

```

import csv

# Vytvoření bodové vrstvy z CSV
csv_file = "souradnice.csv" # Formát: x,y,nazev
output_fc = "Body_z_CSV.shp"

arcpy.management.CreateFeatureclass(
    arcpy.env.workspace,

```

```

        output_fc,
        "POINT",
        spatial_reference=arcpy.SpatialReference(5514)
    )

    arcpy.management.AddField(output_fc, "NAZEV", "TEXT", field_length=50)

    # Čtení a vkládání dat
    with open(csv_file, 'r', encoding='utf-8') as f:
        reader = csv.DictReader(f)

        with arcpy.da.InsertCursor(output_fc, ["SHAPE@XY", "NAZEV"]) as cursor:
            for row in reader:
                x = float(row['x'])
                y = float(row['y'])
                nazev = row['nazev']
                cursor.insertRow([(x, y), nazev])

```

3.2 Tvorba liniové geometrie

Linie vytvoříme z **posloupnosti bodů**. Minimálně dva body jsou potřeba pro vytvoření linie.

3.2.1 Jednoduchá linie

```

# Vytvoření linie ze seznamu bodů
body = [
    arcpy.Point(500000, 6000000),
    arcpy.Point(510000, 6010000),
    arcpy.Point(520000, 6005000),
    arcpy.Point(530000, 6015000)
]

# Body musíme vložit do Array
array = arcpy.Array(body)

# Vytvoření Polyline geometrie
linie = arcpy.Polyline(array)

print(f"Délka linie: {linie.length:.2f} m")

```

3.2.2 Vložení linie do feature class

```
# Vytvoření liniové vrstvy
output_fc = "Nove_linie.shp"

arcpy.management.CreateFeatureclass(
    arcpy.env.workspace,
    output_fc,
    "POLYLINE",
    spatial_reference=arcpy.SpatialReference(5514)
)

arcpy.management.AddField(output_fc, "NAZEV", "TEXT", field_length=50)

# Definice několika linií
linie_data = [
    {
        "nazev": "Linie 1",
        "body": [(500000, 6000000), (510000, 6010000), (520000, 6005000)]
    },
    {
        "nazev": "Linie 2",
        "body": [(505000, 6005000), (515000, 6015000), (525000, 6010000)]
    }
]

with arcpy.da.InsertCursor(output_fc, ["SHAPE@", "NAZEV"]) as cursor:
    for line_dict in lineie_data:
        # Vytvoření bodů z tuples
        body = [arcpy.Point(x, y) for x, y in lineie_dict["body"]]

        # Vytvoření Array z bodů
        array = arcpy.Array(body)

        # Vytvoření Polyline geometrie
        polyline = arcpy.Polyline(array)

        cursor.insertRow([polyline, lineie_dict["nazev"]])
```

3.2.3 Multipart linie

Pokud chceme vytvořit linii s více oddělenými částmi:

```
# Vytvoření multipart linie
cast1 = [
    arcpy.Point(500000, 6000000),
    arcpy.Point(510000, 6010000)
]

cast2 = [
    arcpy.Point(520000, 6020000),
    arcpy.Point(530000, 6030000)
]

# Každá část je samostatný Array
array1 = arcpy.Array(cast1)
array2 = arcpy.Array(cast2)

# Array Arrays pro multipart
parts = arcpy.Array([array1, array2])

# Vytvoření multipart linie
multipart_linie = arcpy.Polyline(parts)

print(f"Počet částí: {multipart_linie.partCount}")
```

3.3 Tvorba polygonové geometrie

Polygon vytvoříme podobně jako linii, ale body musí tvořit **uzavřený tvar** (první a poslední bod by měly být totožné).

3.3.1 Jednoduchý polygon

```
# Vytvoření čtverce
body = [
    arcpy.Point(500000, 6000000),
    arcpy.Point(510000, 6000000),
    arcpy.Point(510000, 6010000),
    arcpy.Point(500000, 6010000),
```

```

    arcpy.Point(500000, 6000000) # Uzavření - stejný jako první bod
]

array = arcpy.Array(body)
polygon = arcpy.Polygon(array)

print(f"Plocha: {polygon.area:.2f} m²")
print(f"Obvod: {polygon.length:.2f} m")

```

Uzavření polygonu

ArcPy automaticky uzavře polygon, i když nezádáte poslední bod stejný jako první. Pro přehlednost je ale doporučeno polygon explicitně uzavřít.

3.3.2 Vložení polygonu do feature class

```

# Vytvoření polygonové vrstvy
output_fc = "Nove_polygon.shp"

arcpy.management.CreateFeatureclass(
    arcpy.env.workspace,
    output_fc,
    "POLYGON",
    spatial_reference=arcpy.SpatialReference(5514)
)

arcpy.management.AddField(output_fc, "NAZEV", "TEXT", field_length=50)
arcpy.management.AddField(output_fc, "PLOCHA", "DOUBLE")

# Vytvoření několika polygonů
polygons = [
    {
        "nazev": "Čtverec",
        "body": [(500000, 6000000), (510000, 6000000),
                  (510000, 6010000), (500000, 6010000),
                  (500000, 6000000)]
    },
    {
        "nazev": "Trojúhelník",
        "body": [(520000, 6000000), (530000, 6000000),

```

```

        (525000, 6010000), (520000, 6000000)]
    }
]

with arcpy.da.InsertCursor(output_fc, ["SHAPE@", "NAZEV", "PLOCHA"]) as cursor:
    for poly_dict in polygony:
        # Vytvoření bodů
        body = [arcpy.Point(x, y) for x, y in poly_dict["body"]]

        # Vytvoření Array a Polygon
        array = arcpy.Array(body)
        polygon = arcpy.Polygon(array)

        cursor.insertRow([polygon, poly_dict["nazev"], polygon.area])

```

3.3.3 Polygon s dírou (vnitřním ringem)

```

# Vnější hranice (velký čtverec)
outer_ring = [
    arcpy.Point(500000, 6000000),
    arcpy.Point(520000, 6000000),
    arcpy.Point(520000, 6020000),
    arcpy.Point(500000, 6020000),
    arcpy.Point(500000, 6000000)
]

# Vnitřní hranice (malý čtverec - díra)
inner_ring = [
    arcpy.Point(505000, 6005000),
    arcpy.Point(515000, 6005000),
    arcpy.Point(515000, 6015000),
    arcpy.Point(505000, 6015000),
    arcpy.Point(505000, 6005000)
]

# Vytvoření Arrays
outer_array = arcpy.Array(outer_ring)
inner_array = arcpy.Array(inner_ring)

# Kombinace - outer ring, None (oddělovač), inner ring

```

```

outer_array.add(None) # Signalizuje začátek vnitřního ringu
outer_array.extend(inner_array)

# Vytvoření polygonu s dírou
polygon_s_dirou = arcpy.Polygon(outer_array)

print(f"Plocha s dírou: {polygon_s_dirou.area:.2f} m2")

```

3.3.4 Multipart polygon

```

# První polygon
poly1_body = [
    arcpy.Point(500000, 6000000),
    arcpy.Point(510000, 6000000),
    arcpy.Point(510000, 6010000),
    arcpy.Point(500000, 6010000),
    arcpy.Point(500000, 6000000)
]

# Druhý polygon (oddělený)
poly2_body = [
    arcpy.Point(520000, 6020000),
    arcpy.Point(530000, 6020000),
    arcpy.Point(530000, 6030000),
    arcpy.Point(520000, 6030000),
    arcpy.Point(520000, 6020000)
]

# Vytvoření samostatných Arrays
array1 = arcpy.Array(poly1_body)
array2 = arcpy.Array(poly2_body)

# Array Arrays
parts = arcpy.Array([array1, array2])

# Vytvoření multipart polygonu
multipart_polygon = arcpy.Polygon(parts)

print(f"Počet částí: {multipart_polygon.partCount}")
print(f"Celková plocha: {multipart_polygon.area:.2f} m2")

```

3.4 Úlohy

Úloha 17.6. Vytvořte bodovou vrstvu s 10 náhodnými body v rozsahu souřadnic 500000-600000 (X) a 6000000-6100000 (Y). Použijte modul `random`.

Úloha 17.7. Napište skript, který vytvoří linii ve tvaru písmene “Z” a uloží ji do nové feature class.

Úloha 17.8. Vytvořte polygon ve tvaru pravidelného pětiúhelníku se středem v bodě [550000, 6050000] a poloměrem 10000 m. Použijte `math.cos()` a `math.sin()` pro výpočet souřadnic vrcholů.

Úloha 17.9. Napište funkci, která přečte souřadnice z textového souboru (každý řádek obsahuje x,y) a vytvoří z nich linii.

Úloha 17.10. Vytvořte polygonovou vrstvu s mřížkou (grid) 3×3 čtverců o velikosti 5000×5000 m. Začátek mřížky v bodě [500000, 6000000].

4 Tahák - časté vzory použití

4.1 Čtení geometrií

4.1.1 Výpis všech souřadnic bodové vrstvy

```
with arcpy.da.SearchCursor("Body.shp", ["SHAPE@XY", "NAZEV"]) as cursor:
    for row in cursor:
        x, y = row[0]
        print(f"{row[1]}: [{x:.2f}, {y:.2f}]")
```

4.1.2 Počítání lomových bodů linie

```
with arcpy.da.SearchCursor("Linie.shp", ["SHAPE@"]) as cursor:
    for row in cursor:
        line = row[0]
        total = sum(len([p for p in part if p]) for part in line)
        print(f"Celkem bodů: {total}")
```

4.1.3 Extrakce hraničních bodů polygonu

```
with arcpy.da.SearchCursor("Polygony.shp", ["SHAPE@", "NAZEV"]) as cursor:
    for row in cursor:
        polygon = row[0]
        extent = polygon.extent
        print(f"{row[1]}: [{extent.XMin:.0f}, {extent.YMin:.0f}] - [{extent.XMax:.0f}, {extent.YMax:.0f}"])
```

4.2 Vytváření geometrií

4.2.1 Vytvoření bodu pomocí SHAPE@XY

```
with arcpy.da.InsertCursor("Body.shp", ["SHAPE@XY", "NAZEV"]) as cursor:
    cursor.insertRow([(500000, 6000000), "Nový bod"])
```

4.2.2 Vytvoření bodu pomocí Point objektu

```
point = arcpy.Point(500000, 6000000)
point_geom = arcpy.PointGeometry(point)

with arcpy.da.InsertCursor("Body.shp", ["SHAPE@", "NAZEV"]) as cursor:
    cursor.insertRow([point_geom, "Nový bod"])
```

4.2.3 Vytvoření linie ze seznamu souřadnic

```
coords = [(500000, 6000000), (510000, 6010000), (520000, 6005000)]
body = [arcpy.Point(x, y) for x, y in coords]
array = arcpy.Array(body)
linie = arcpy.Polyline(array)

with arcpy.da.InsertCursor("Linie.shp", ["SHAPE@"]) as cursor:
    cursor.insertRow([linie])
```

4.2.4 Vytvoření polygonu ze seznamu souřadnic

```
coords = [(500000, 6000000), (510000, 6000000), (510000, 6010000),
          (500000, 6010000), (500000, 6000000)]
body = [arcpy.Point(x, y) for x, y in coords]
array = arcpy.Array(body)
polygon = arcpy.Polygon(array)

with arcpy.da.InsertCursor("Polygony.shp", ["SHAPE@"]) as cursor:
    cursor.insertRow([polygon])
```

4.3 Konverze mezi geometriemi

4.3.1 Převod centroidů polygonů na body

```
arcpy.management.CreateFeatureclass(
    arcpy.env.workspace, "Centroidy.shp", "POINT"
)

with arcpy.da.SearchCursor("Polygony.shp", ["SHAPE@", "NAZEV"]) as search:
    with arcpy.da.InsertCursor("Centroidy.shp", ["SHAPE@XY", "NAZEV"]) as insert:
        for row in search:
            centroid = row[0].centroid
            insert.insertRow([(centroid.X, centroid.Y), row[1]])
```

4.3.2 Převod linií na počáteční a koncové body

```
arcpy.management.CreateFeatureclass(
    arcpy.env.workspace, "Koncove_body.shp", "POINT"
)
arcpy.management.AddField("Koncove_body.shp", "TYP", "TEXT", field_length=10)

with arcpy.da.SearchCursor("Linie.shp", ["SHAPE@"]) as search:
    with arcpy.da.InsertCursor("Koncove_body.shp", ["SHAPE@XY", "TYP"]) as insert:
        for row in search:
            line = row[0]
            first = line.firstPoint
```

```

last = line.lastPoint

insert.insertRow([(first.X, first.Y), "START"])
insert.insertRow([(last.X, last.Y), "END"])

```

4.4 Import dat

4.4.1 Import bodů z CSV

```

import csv

with open("data.csv", 'r') as f:
    reader = csv.DictReader(f)
    with arcpy.da.InsertCursor("Body.shp", ["SHAPE@XY", "NAZEV"]) as cursor:
        for row in reader:
            x, y = float(row['x']), float(row['y'])
            cursor.insertRow([(x, y), row['nazev']])

```

4.4.2 Import linií z textového souboru

```

# Formát souboru: každý řádek je jeden bod linie "x,y"
# Prázdný řádek znamená novou linii

with open("linie.txt", 'r') as f:
    body = []

    with arcpy.da.InsertCursor("Linie.shp", ["SHAPE@"]) as cursor:
        for line in f:
            line = line.strip()

            if line: # Neprázdný řádek - přidat bod
                x, y = map(float, line.split(','))
                body.append(arcpy.Point(x, y))
            else: # Prázdný řádek - uložit linii
                if len(body) >= 2:
                    array = arcpy.Array(body)
                    polyline = arcpy.Polyline(array)
                    cursor.insertRow([polyline])

```

```

        body = []

    # Uložení poslední linie
    if len(body) >= 2:
        array = arcpy.Array(body)
        polyline = arcpy.Polyline(array)
        cursor.insertRow([polyline])

```

4.5 Generování geometrií

4.5.1 Vytvoření pravidelné mřížky bodů

```

# Parametry mřížky
x_start, y_start = 500000, 6000000
spacing = 5000
rows, cols = 5, 5

with arcpy.da.InsertCursor("Mrizka.shp", ["SHAPE@XY", "ID"]) as cursor:
    point_id = 1
    for row in range(rows):
        for col in range(cols):
            x = x_start + col * spacing
            y = y_start + row * spacing
            cursor.insertRow([(x, y), point_id])
            point_id += 1

```

4.5.2 Vytvoření pravidelného mnohoúhelníku

```

import math

def create_regular_polygon(center_x, center_y, radius, num_vertices):
    """Vytvoří pravidelný mnohoúhelník"""
    body = []

    for i in range(num_vertices + 1): # +1 pro uzavření
        angle = 2 * math.pi * i / num_vertices
        x = center_x + radius * math.cos(angle)
        y = center_y + radius * math.sin(angle)

```

```

        body.append(arcpy.Point(x, y))

    array = arcpy.Array(body)
    return arcpy.Polygon(array)

# Použití
polygon = create_regular_polygon(550000, 6050000, 10000, 6) # šestiúhelník

with arcpy.da.InsertCursor("Polygony.shp", ["SHAPE@"]) as cursor:
    cursor.insertRow([polygon])

```

4.5.3 Vytvoření mřížky polygonů

```

def create_grid_cell(x_min, y_min, cell_size):
    """Vytvoří jeden čtvercový polygon"""
    body = [
        arcpy.Point(x_min, y_min),
        arcpy.Point(x_min + cell_size, y_min),
        arcpy.Point(x_min + cell_size, y_min + cell_size),
        arcpy.Point(x_min, y_min + cell_size),
        arcpy.Point(x_min, y_min)
    ]
    return arcpy.Polygon(arcpy.Array(body))

# Vytvoření mřížky
x_start, y_start = 500000, 6000000
cell_size = 5000
grid_rows, grid_cols = 3, 3

with arcpy.da.InsertCursor("Mrizka.shp", ["SHAPE@", "ID"]) as cursor:
    cell_id = 1
    for row in range(grid_rows):
        for col in range(grid_cols):
            x = x_start + col * cell_size
            y = y_start + row * cell_size
            polygon = create_grid_cell(x, y, cell_size)
            cursor.insertRow([polygon, cell_id])
            cell_id += 1

```

4.6 Zpracování chyb

4.6.1 Kontrola typu geometrie

```
with arcpy.da.SearchCursor("Vrstva.shp", ["SHAPE@"]) as cursor:
    for row in cursor:
        geom = row[0]

        if geom.type == "point":
            print("Bod")
        elif geom.type == "polyline":
            print("Linie")
        elif geom.type == "polygon":
            print("Polygon")
```

4.6.2 Ošetření neplatných geometrií

```
with arcpy.da.SearchCursor("Polygony.shp", ["SHAPE@", "OID@"]) as cursor:
    for row in cursor:
        geom = row[0]

        if geom is None:
            print(f"Prázdná geometrie u záznamu {row[1]}")
            continue

        if geom.area == 0:
            print(f"Nulová plocha u záznamu {row[1]}")
```

4.6.3 Kontrola uzavření polygonu

```
def is_closed(polygon):
    """Zkontroluje, zda je polygon uzavřený"""
    for part in polygon:
        points = [p for p in part if p is not None]
        if len(points) < 3:
            return False
```

```

        first = points[0]
        last = points[-1]

        if first.X != last.X or first.Y != last.Y:
            return False

    return True

with arcpy.da.SearchCursor("Polygony.shp", ["SHAPE@", "OID@"]) as cursor:
    for row in cursor:
        if not is_closed(row[0]):
            print(f"Neuzavřený polygon: záznam {row[1]}")

```

4.7 Práce se souřadnicovými systémy

4.7.1 Specifikace SR při vytváření feature class

```

# S-JTSK (EPSG:5514)
sr_jtsk = arcpy.SpatialReference(5514)

arcpy.management.CreateFeatureclass(
    arcpy.env.workspace,
    "Body_JTSK.shp",
    "POINT",
    spatial_reference=sr_jtsk
)

```

4.7.2 Transformace souřadnic

```

# Vytvoření geometrie v jednom SR
point_wgs = arcpy.PointGeometry(
    arcpy.Point(-15.0, 50.0),
    arcpy.SpatialReference(4326) # WGS-84
)

# Transformace do jiného SR
point_jtsk = point_wgs.projectAs(arcpy.SpatialReference(5514)) # S-JTSK

```

```
print(f"WGS-84: {point_wgs.firstPoint.X}, {point_wgs.firstPoint.Y}")  
print(f"S-JTSK: {point_jtsk.firstPoint.X}, {point_jtsk.firstPoint.Y}")
```